

Application Development

Lecture 2: The C# Programming Language

Konstantin Kapinchev
University of Greenwich

Application Development

Lecture 2: The C# Programming Language

In this lecture:

- **Historical Overview**
- **Programming Paradigms**
- **Variables and Constants**
- **Data Types**
- **Logical Expressions**
- **Control Flow Statements**
- **Arrays**
- **Conclusions**

Application Development

Lecture 2: The C# Programming Language

Historical Overview

- In the mid 1960s, an operating system called UNIX¹ was introduced
- Initially, it was written in assembly Language²
- In 1973, UNIX Version 4 was written in the C programming language
- This language was designed with the main purpose to improve the portability of UNIX

¹An attempt to standardise UNIX and other related OS, including macOS, resulted in POSIX

²Software written in assembly language has limited portability

Application Development

Lecture 2: The C# Programming Language

Historical Overview *continued*

- In 1985, the first ANSI C standard was released
- In the same year, a C-like programming language, called C++, was released
- One of its main new features was the object-oriented functionality
- In 2000, Microsoft released a new language called C#
- It is based on C++, which combined features inspired by Java¹

¹Java was released in 1996 by Sun Microsystems

Application Development

Lecture 2: The C# Programming Language

Programming Paradigms

- **C# is a multi-paradigm programming language**
 - **The most notable of those paradigms are:**
 - **Structured¹**
 - **Object-oriented²**
 - **Event-driven³**
 - **Functional**
 - **Reflective**
 - **Concurrent**

¹Control Flow Statements

²Classes and Objects

³User interaction with Graphical User Interface

Application Development

Lecture 2: The C# Programming Language

Variables and Constants

- **Variables**
 - A memory location, which can store a single value
 - It has:
 - Name
 - Memory Address¹
 - Data Type
 - Value
 - Scope

¹Not within the scope of this module

Application Development

Lecture 2: The C# Programming Language

Variables and Constants *continued*

- Declare/define variables

Type of the variable	←	// Example:
Name of the variable	←	int MyVariable;
Value	←	int MyNewVariable = 5;
New value provided at a later stage	←	MyVariable = 10;
Last value will be displayed	←	MyNewVariable = 15;
		Console.WriteLine("{0} {1}", MyVariable, MyNewVariable);

Application Development

Lecture 2: The C# Programming Language

Variables and Constants *continued*

- Constants
 - Unlike variables, constants cannot change their values

```
// Example:  
const int MyConstant = 5;  
  
// Will generate an error  
MyConstant = 10;
```

Key word

Type of the constant

Name of the constant

This assignment statement will generate an error, since constants, once declared, cannot change their values

Application Development

Lecture 2: The C# Programming Language

Data Types

- Some of the most popular data types in C# are:
 - Integer number, keyword *int*
 - Real number, keyword *double*
 - Boolean, keyword *bool*
 - Character, keyword *char*
 - String/Text, keyword *string*

Application Development

Lecture 2: The C# Programming Language

Data Types *continued*

- Variables
 - The following source code fragment initialises variables of five different types and displays their values on the screen

// Example:

```
int IntegerNumber = 5;  
double RealNumber = 5.5;  
char Symbol = 'A';  
string Text = "Hello";  
bool Flag = true;  
  
Console.WriteLine(IntegerNumber);  
Console.WriteLine(RealNumber);  
Console.WriteLine(Symbol);  
Console.WriteLine(Text);  
Console.WriteLine(Flag);
```

Application Development

Lecture 2: The C# Programming Language

Data Types *continued*

- Variables
 - The types of the variables can be obtained by using the "GetType" function

// Example:

```
int IntegerNumber = 5;  
double RealNumber = 5.5;  
char Symbol = 'A';  
string Text = "Hello";  
bool Flag = true;
```

```
Console.WriteLine(IntegerNumber.GetType());  
Console.WriteLine(RealNumber.GetType());  
Console.WriteLine(Symbol.GetType());  
Console.WriteLine(Text.GetType());  
Console.WriteLine(Flag.GetType());
```

Application Development

Lecture 2: The C# Programming Language

Data Types *continued*

- Type conversion
 - In some cases, it is possible to change from one type into another, as demonstrated in this example

// Example:

```
int IntegerNumber = 5;  
double RealNumber = 5.5;
```

```
RealNumber = (double)IntegerNumber;  
IntegerNumber = (int)5.55;
```

```
Console.WriteLine("Integer value: {0}", IntegerNumber);  
Console.WriteLine("Integer value: {0} ", RealNumber);
```

Application Development

Lecture 2: The C# Programming Language

Logical Expressions

- Also called Boolean¹ Expressions
- Their value can be either true or false (type Boolean)
- They may consist of:
 - Boolean variables and constants
 - Boolean operations (OR, AND, NOT and others)
 - Comparison Operators

Comparison Operators	
==	Equal
!=	Not equal
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to

¹Named after the mathematician George Boole

Application Development

Lecture 2: The C# Programming Language

Logical Expressions *continued*

- The following example evaluates a logical expression, based on a user input
- The value of the logical expression is assigned to a Boolean variable Flag
- The value of the logical expression, which can be either *true* or *false*, is displayed alongside the user input

// Example:

```
int UserInput;  
bool Flag;
```

```
Console.WriteLine("Enter a value:");  
UserInput = Convert.ToInt32(Console.ReadLine());
```

```
Flag = (UserInput > 0);  
Console.WriteLine("The value is positive, {0}", Flag);
```

Application Development

Lecture 2: The C# Programming Language

Control Flow Statements

- **Statements which determine the path of the programs**
- **The statements make a choice between two or more possible paths**
- **Can repeat a block of statements multiple times, based on a condition**
- **The condition can be implemented by using logical expression**

Application Development

Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- The following are some of the most popular control flow statements:
 - Subroutines
 - Conditional Statements
 - IF statement
 - SWITCH statement
 - Loop Statements
 - FOR Loop
 - WHILE loop

Application Development

Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- IF Statement
 - The IF statement chooses one of two possible paths, based on the value of a logical expression, which can be either *true* or *false*¹
 - The statement can be nested, which results in more than two possible paths

¹It will have only one path, if the "else" keyword is not used

Application Development

Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- IF statement

- Example:

```
int UserInput;
Console.WriteLine("Enter a value:");
UserInput = Convert.ToInt32(Console.ReadLine());
if (UserInput > 0)
    Console.WriteLine("The value is positive");
else
    Console.WriteLine("The value is negative or 0");
```

Application Development

Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- **FOR Loop**
 - The FOR loop repeats a *block of statements*, based on the value of an index
 - There are 3 statements after keyword FOR, which control its iterations:
 - Statement 1: runs once before the *block of statements*
 - Statement 2: defines a condition for exiting the loop
 - Statement 3: runs every time after the *block of statements*

Application Development

Lecture 2: The C# Programming Language

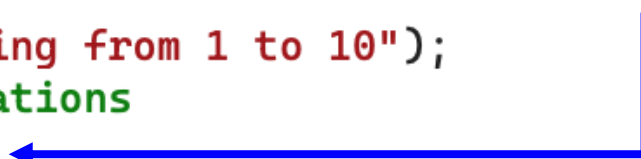
Control Flow Statements *continued*

- FOR Loop

- Example:

```
// Declare index
int i;
// Display text
Console.WriteLine("Counting from 1 to 10");
// FOR loop with 10 iterations
for (i = 0; i < 10; i++)
{
    Console.WriteLine(i + 1);
}
```

*Statements
controlling the
iterations of
the FOR loop*



Application Development

Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- **WHILE Loop**
 - The WHILE loop repeats a block of statements, based on the value of a logical expression
 - As long as the value of the logical expression is *true*, the WHILE loop will repeat its block of statements
 - Once the value of the logical expression is *false*, the WHILE loop will exit

Application Development

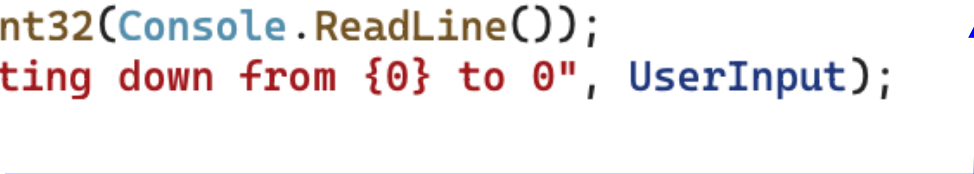
Lecture 2: The C# Programming Language

Control Flow Statements *continued*

- WHILE Loop

- Example:

```
int UserInput;
Console.WriteLine("Enter a value:");
UserInput = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Counting down from {0} to 0", UserInput);

while (UserInput >= 0)
{
    Console.WriteLine(UserInput);
    UserInput = UserInput - 1;
}
```
- Logical expression*
- 

Application Development

Lecture 2: The C# Programming Language

Arrays

- **Overview**
 - **One of the most popular data structures**
 - **An array is a collection of elements of the same data type**
 - **Each element can be accessed by an index**
 - **In most programming languages, C# included, the index starts from 0**

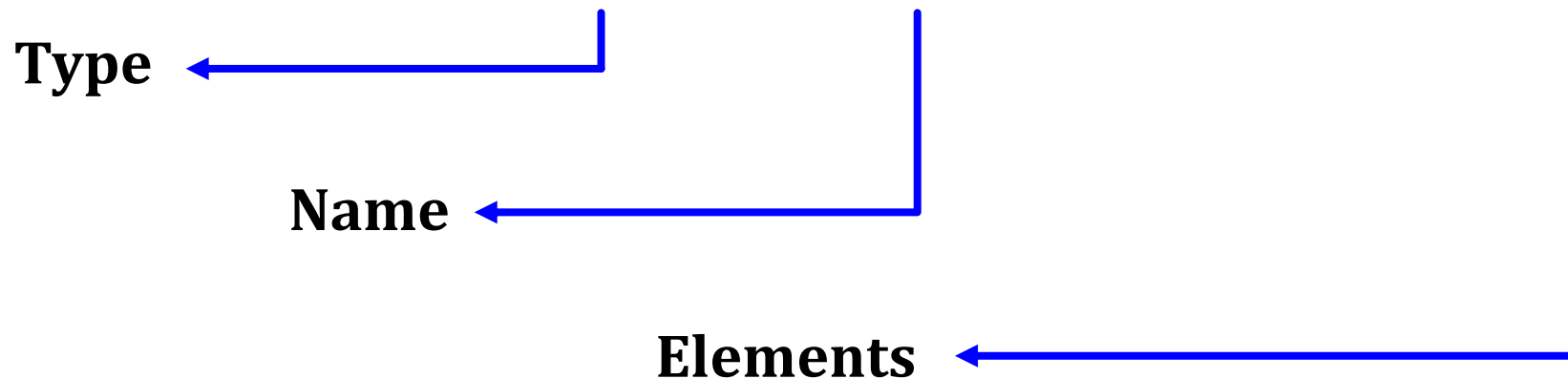
Application Development

Lecture 2: The C# Programming Language

Arrays continued

- C# Array Syntax
 - Declare an array:

```
string[] MyArray = { "Apple", "Orange", "Apricot" };
```



Application Development

Lecture 2: The C# Programming Language

Arrays continued

- C# Array Syntax
 - Access the first array element:

```
Console.WriteLine("The first element is: {0}", MyArray[0]);
```

Array Name

Index (starting from 0)

Application Development

Lecture 2: The C# Programming Language

Arrays continued

- C# Array Syntax
 - Visualise the entire array
 - Using "foreach":

```
foreach (string S in MyArray)
{
    Console.WriteLine(S);
}
```

Application Development

Lecture 2: The C# Programming Language

Arrays continued

- C# Array Syntax
 - Visualise the entire array

- Using FOR loop:

```
// Declare index
int i;
// Loop displaying all elements with corresponding indices
for(i = 0; i < 3; i++)
{
    Console.WriteLine("{0}. {1}", i + 1, MyArray[i]);
}
```

Result:

1. Apple
2. Orange
3. Apricot

Application Development

Lecture 2: The C# Programming Language

Conclusions

- **This lecture discussed key features of the C# programming language**
- **Those include data types, variables, arrays and control flow statements**
- **Source code fragments illustrated those key features**
- **Next lecture will introduce the Object-Oriented Programming (OOP) and will take a closer look at classes and objects**